Student Marks Matrix Analysis

Analyse marks across multiple subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

Problem Statement

Develop a Python program to analyse a matrix of student marks for different subjects. Calculate each student's total, average, highest and lowest marks. Identify the top performer(s) based on total/average marks. Calculate the average marks for each subject and detect subjects with low average performance (e.g., average below a given threshold).

Python Code

```
# Student Marks Matrix Analysis
students = ["Arun", "Bhanu", "Charan", "Divya", "Esha"]
subjects = ["Python", "C", "Mathematics", "English"]
# Marks of students in different subjects
marks = [
    [85, 78, 92, 88],
    [72, 80, 75, 70],
    [90, 95, 89, 92],
    [40, 45, 35, 50],
    [65, 70, 60, 75]
]
number_of_students = len(students)
number_of_subjects = len(subjects)

# Calculate student-wise statistics
totals = []
averages = []
print("STUDENT-WISE ANALYSIS")
print("-" * 45)
for i in range(number_of_students):
    total = sum(marks[i])
    average = total / number_of_subjects
    totals.append(total)
    averages.append(average)
    print(students[i], "Total =", total, "Average =", round(average, 2))
# Identify the top-performing student
highest_total = max(totals)
top_index = totals.index(highest_total)
print("\nTOP-PERFORMING STUDENT")
print("Name:", students[top_index])
print("Total Marks:", highest_total)

# Calculate subject-wise statistics
print("\nSUBJECT-WISE ANALYSIS")
print("-" * 45)
low_performing_subjects = []
for j in range(number_of_subjects):
    subject_total = 0
    for i in range(number_of_students):
        subject_total += marks[i][j]
    subject_average = subject_total / number_of_students
    print(subjects[j], "Average =", round(subject_average, 2))
    if subject_average < 50:
        low_performing_subjects.append(subjects[j])
# Display low-performing subjects
print("\nLOW-PERFORMING SUBJECTS")
if len(low_performing_subjects) > 0:
    for subject in low_performing_subjects:
        print(subject)
else:
    print("No low-performing subjects")

print("\nAnalysis completed successfully.")
```

Program Output

```
STUDENT-WISE ANALYSIS
---------------------------------------------
Arun     Total = 343   Average = 85.75
Bhanu    Total = 297   Average = 74.25
Charan   Total = 366   Average = 91.5
Divya    Total = 170   Average = 42.5
Esha     Total = 280   Average = 70.0

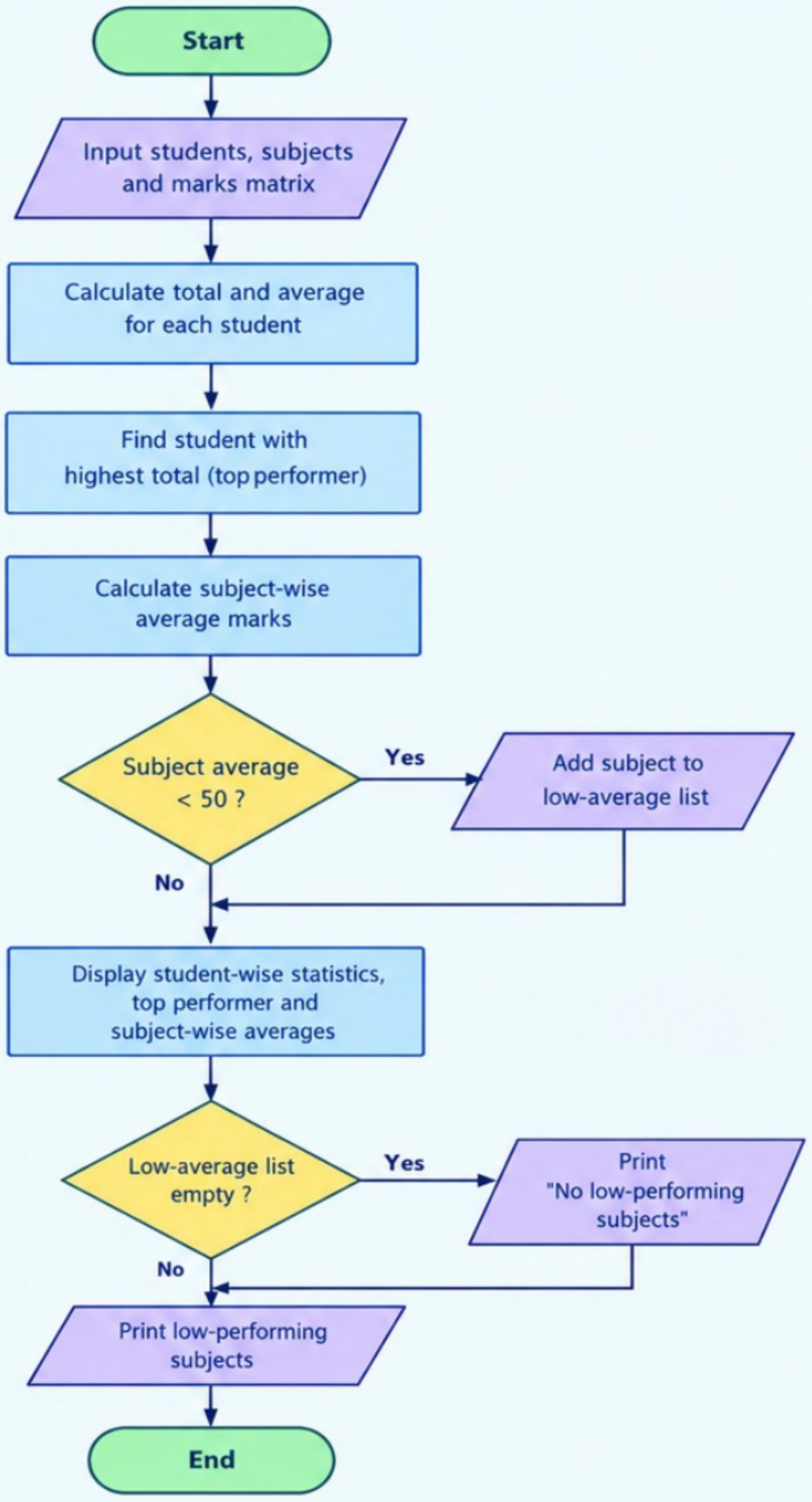
TOP-PERFORMING STUDENT
Name: Charan
Total Marks: 366

SUBJECT-WISE ANALYSIS
---------------------------------------------
Python         Average = 74.0
C              Average = 72.0
Mathematics    Average = 75.0
English        Average = 69.0

LOW-PERFORMING SUBJECTS
No low-performing subjects

Analysis completed successfully.
```

Flowchart



Progressive Electricity Billing

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

Problem Statement

Write a Python program to calculate household electricity bills using multiple consumption slabs, fixed charges, surcharges, and rebates. The program should compute the bill for several months, compare the bills, and identify any unusual increases in the bill amount.

Python Code

```
# Progressive Electricity Billing System
months = ["January", "February", "March", "April"]
bills = []

fixed_charge = 50
surcharge_rate = 5
rebate_rate = 2

for month in months:
    units = int(input("Enter electricity units consumed in " + month + ": "))

    # Calculate charges using progressive slabs
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = (100 * 2) + (units - 100) * 3
    elif units <= 300:
        energy_charge = (100 * 2) + (100 * 3) + (units - 200) * 5
    else:
        energy_charge = (100 * 2) + (100 * 3) + (100 * 5) + (units - 300) * 7

    # Calculate surcharge and rebate
    surcharge = energy_charge * surcharge_rate / 100
    rebate = energy_charge * rebate_rate / 100

    # Calculate total bill
    total_bill = energy_charge + fixed_charge + surcharge - rebate
    bills.append(total_bill)

    print("\nMonth:", month)
    print("Units Consumed:", units)
    print("Energy Charge: Rs.", round(energy_charge, 2))
    print("Fixed Charge: Rs.", fixed_charge)
    print("Surcharge: Rs.", round(surcharge, 2))
    print("Rebate: Rs.", round(rebate, 2))
    print("Total Electricity Bill: Rs.", round(total_bill, 2))

# Compare bills across months
print("\n--- Monthly Bill Comparison ---")
for i in range(len(months)):
    print(months[i], ": Rs.", round(bills[i], 2))

# Identify unusual increases
for i in range(1, len(bills)):
    increase = bills[i] - bills[i - 1]
    if increase > 0:
        percentage = (increase / bills[i - 1]) * 100
        if percentage > 25:
            print("\nWarning: Unusual bill increase detected!")
            print("From", months[i - 1], "to", months[i])
            print("Increase: Rs.", round(increase, 2))
            print("Percentage Increase:", round(percentage, 2), "%")
        else:
            print("\nBill increase from", months[i - 1], "to", months[i], "is", round(percentage, 2), "%")
    elif increase < 0:
        print("\nBill decreased from", months[i - 1], "to", months[i])
    else:
        print("\nBill remained unchanged from", months[i - 1], "to", months[i])
```

Program Output

```
Enter electricity units consumed in January: 120
Month: January
Units Consumed: 120
Energy Charge: Rs. 260.0
Fixed Charge: Rs. 50
Surcharge: Rs. 13.0
Rebate: Rs. 5.2
Total Electricity Bill: Rs. 317.8

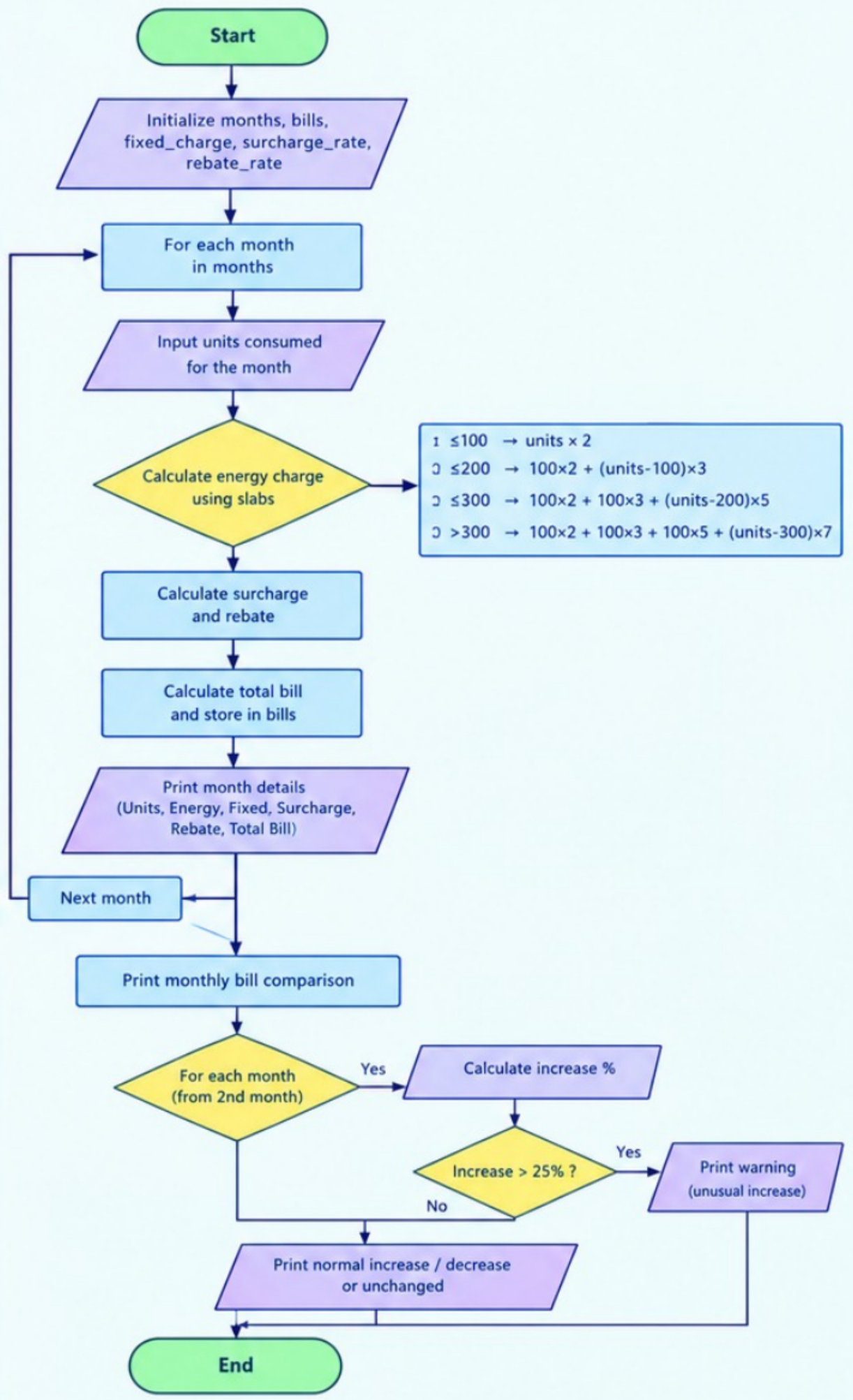
Enter electricity units consumed in February: 180
Month: February
Units Consumed: 180
Energy Charge: Rs. 460.0
Fixed Charge: Rs. 50
Surcharge: Rs. 23.0
Rebate: Rs. 9.2
Total Electricity Bill: Rs. 523.8

Enter electricity units consumed in March: 250
Month: March
Units Consumed: 250
Energy Charge: Rs. 860.0
Fixed Charge: Rs. 50
Surcharge: Rs. 43.0
Rebate: Rs. 17.2
Total Electricity Bill: Rs. 935.8

Enter electricity units consumed in April: 320
Month: April
Units Consumed: 320
Energy Charge: Rs. 1160.0
Surcharge: Rs. 58.0
Rebate: Rs. 23.2
Total Electricity Bill: Rs. 1244.8

--- Monthly Bill Comparison ---
January : Rs. 317.8
February : Rs. 523.8
March : Rs. 935.8
April : Rs. 1244.8
Bill increase from January to February is 64.71 %
Bill increase from February to March is 78.75 %
Warning: Unusual bill increase detected!
From March to April
Increase: Rs. 309.0
Percentage Increase: 33.04 %
```

Flowchart



Student Timetable Conflict Detection:

Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

Problem Statement

The program maintains a student timetable and checks for time conflicts and room availability. It also validates a proposed course to ensure no overlapping periods or unavailable rooms.

Python Code

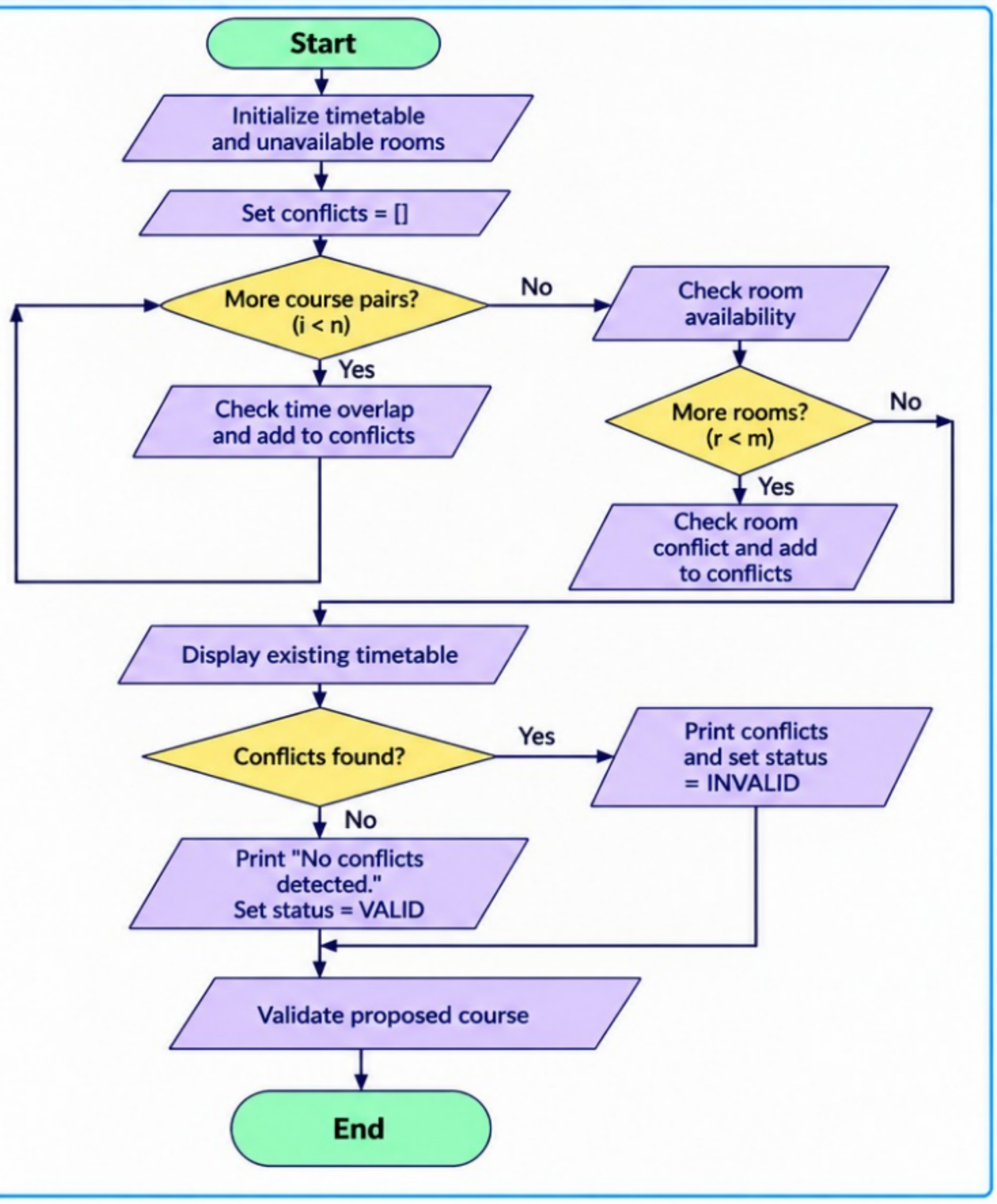
```
# Student Timetable Conflict Detection
# Timetable format: (course, day, start_time, end_time, room)
timetable = [
     ("Python", "Monday", 9, 11, "R101"),
     ("Mathematics", "Monday", 10, 12, "R102"),
     ("Physics", "Tuesday", 9, 11, "R103"),
     ("English", "Wednesday", 11, 12, "R101")
]
# Unavailable rooms and their time periods
unavailable_rooms = [
     ("R101", "Monday", 9, 10),
     ("R103", "Tuesday", 10, 12)
]
conflicts = []
# Detect overlapping course periods
for i in range(len(timetable)):
     for j in range(i + 1, len(timetable)):
          c1 = timetable[i]
          c2 = timetable[j]
          if c1[1] == c2[1]:
               if c1[2] < c2[3] and c2[2] < c1[3]:
                    conflicts.append("Time conflict: " + c1[0] + " and " + c2[0]")
# Check room availability
for course in timetable:
     for room in unavailable_rooms:
          if course[4] == room[0] and course[1] == room[1]:
               if course[2] < room[3] and room[2] < course[3]:
                    conflicts.append("Room conflict: " + course[0] +
                         " uses unavailable room " + course[4]")
# Display existing timetable
print("========== STUDENT TIMETABLE ==========")
for course in timetable:
     print(course[0], " - ", course[1], " Time: ", str(course[2]) + ":00 to",
          str(course[3]) + ":00", " Room: ", course[4])
```

Program Output

```
========= STUDENT TIMETABLE ==========
Python - Monday    Time: 9:00 to 11:00    Room: R101
Mathematics - Monday   Time: 10:00 to  12:00    Room: R102
Physics - Tuesday     Time: 9:00 to 11:00     Room: R103
English - Wednesday   Time: 11:00 to 12:00    Room: R101

========= CONFLICT REPORT =========
Time conflict: Python and Mathematics
Room conflict: Python uses unavailable room R101

Timetable Status: INVALID
```



Warehouse Inventory Reconciliation

Compare opening stock, incoming quantities, dispatched quantities, and closing stock across multiple products; detect discrepancies and calculate replenishment requirements.

Problem Statement

For each product, the expected closing stock is calculated as: Expected Stock = Opening Stock + Incoming – Dispatched. Compare the expected stock with the actual closing stock, determine any discrepancy, and calculate replenishment quantity if the actual stock is below the minimum stock level.

Python Code

```
# Warehouse Inventory Reconciliation
products = []
n = int(input("Enter number of products: "))

for i in range(n):
    print("\nProduct", i + 1)
    name = input("Enter product name: ")
    opening_stock = int(input("Opening stock: "))
    incoming = int(input("Incoming quantity: "))
    dispatched = int(input("Dispatched quantity: "))
    closing_stock = int(input("Actual closing stock: "))
    minimum_stock = int(input("Minimum stock: "))

    expected_stock = opening_stock + incoming - dispatched
    discrepancy = closing_stock - expected_stock
    replenishment = max(0, minimum_stock - closing_stock)

    if discrepancy > 0:
        status = "Excess stock"
    elif discrepancy < 0:
        status = "Stock shortage"
    else:
        status = "Stock matched"

    products.append([name, opening_stock, incoming, dispatched,
                     closing_stock, expected_stock, discrepancy,
                     status, replenishment])

print("\n========== INVENTORY RECONCILIATION REPORT ==========")
print("Product\tOpening\tIncoming\tDispatched\tActual\tExpected\tDiscrepancy\tStatus\tReplenishment")
for p in products:
    print(p[0], "\t", p[1], "\t", p[2], "\t", p[3], "\t", p[4], "\t", p[5], "\t", p[6], "\t", p[7], "\t", p[8])
```

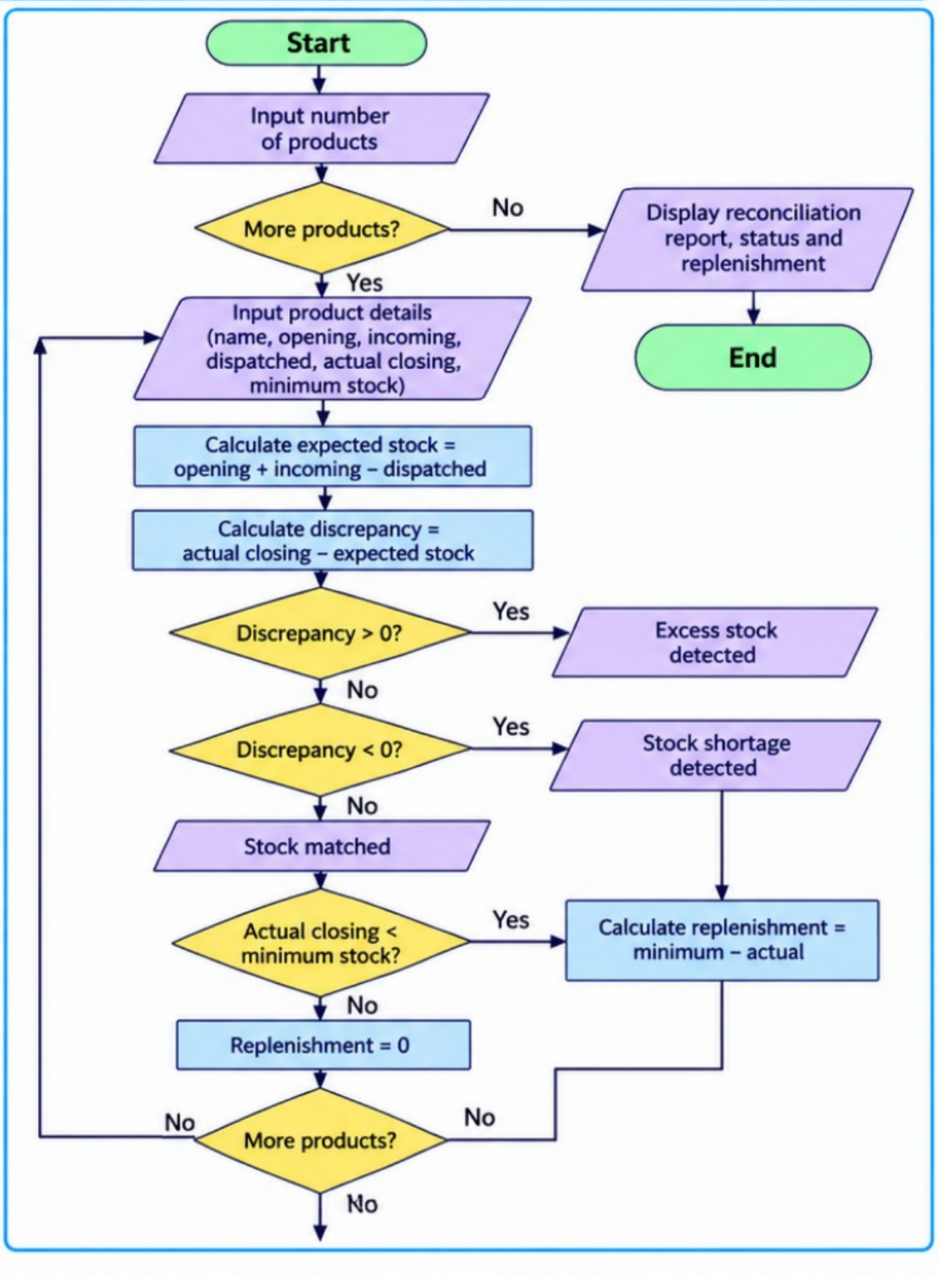
Program Output

```
Enter number of products: 2

Product 1
Enter product name: Laptop
Opening stock: 50
Incoming quantity: 20
Dispatched quantity: 40
Actual closing stock: 35
Minimum stock: 20

Product 2
Enter product name: Mouse
Opening stock: 100
Incoming quantity: 30
Dispatched quantity: 50
Actual closing stock: 70
Minimum stock: 80

========== INVENTORY RECONCILIATION REPORT ==========
Product     Opening   Incoming  Dispatched  Actual   Expected  Discrepancy  Status         Replenishment
Laptop      50        20        40          35       30        5            Excess stock:  0
Mouse       100       30        50          70       80        -10          Stock matched  0
Reconciliation completed.
```



ATM Transaction Sequence Simulation

Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

Problem Statement

Develop a program to process a sequence of deposits and withdrawals, update a simulated account balance, enforce a Rs. 10,000 withdrawal limit, reject invalid amounts (≤ 0), insufficient balance, and invalid choices, and finally display a transaction summary with total deposits, total withdrawals, rejected transactions, and the final balance.

Python Code

```
# ATM Transaction Sequence Simulation
balance = float(input("Enter initial balance: "))
n = int(input("Enter number of transactions:"))

transactions = []
deposits = 0
withdrawals = 0
rejected = 0

withdrawal_limit = 10000

for i in range(n):
    print("\nTransaction", i + 1)
    print("1. Deposit")
    print("2. Withdraw")

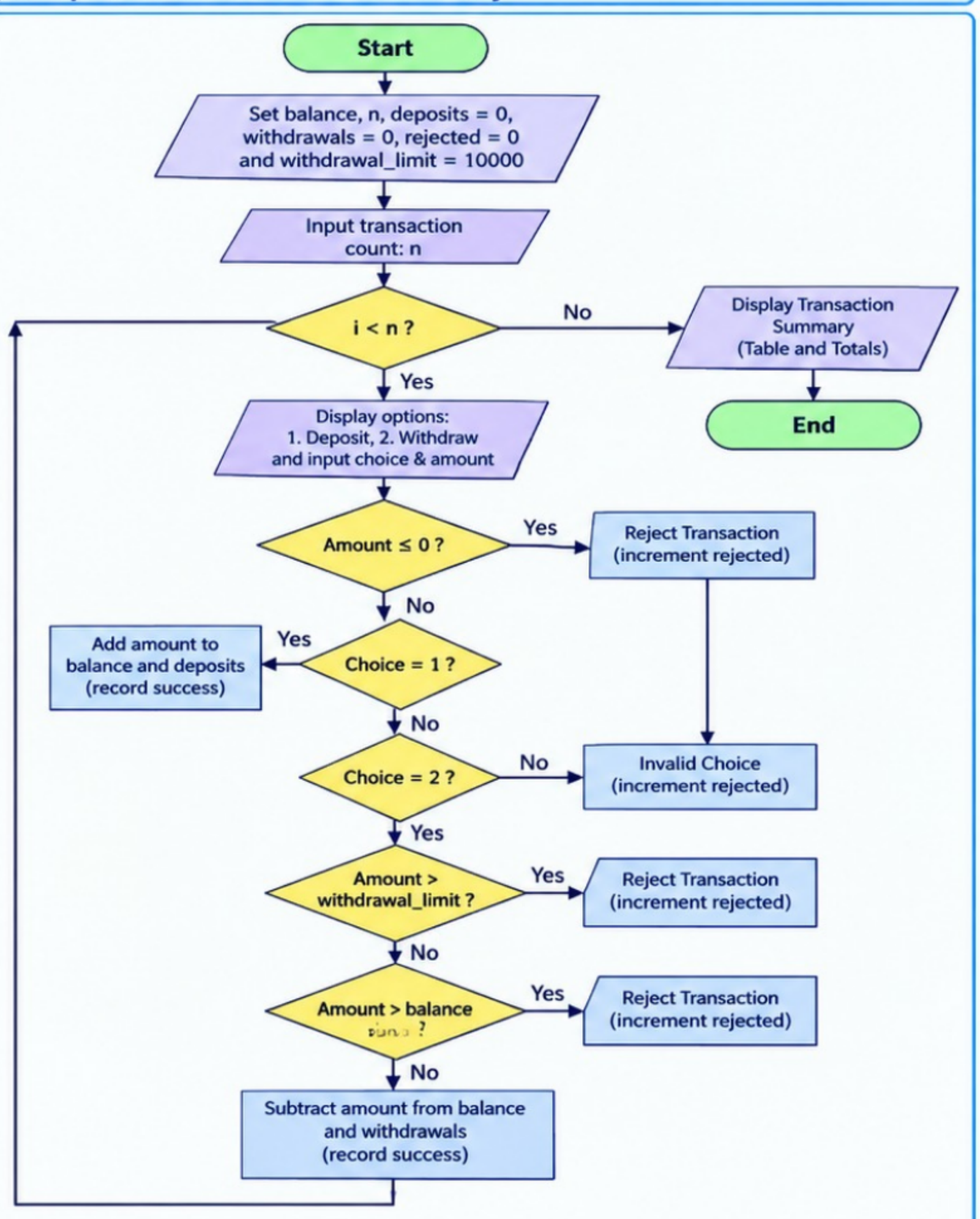
    choice = input("Enter your choice: ")
    amount = float(input("Enter amount: "))
    if amount <= 0:
        print("Invalid amount! Transaction rejected.")
        rejected += 1
    elif choice == "1":
        balance += amount
        deposiis += amount
        transactions.append(["Deposit", amount, "Successful"])
        print("Deposit successful."))
    elif choice == "2":
        if amount > withdrawal_limit:
            print("Withdrawal limit exceeded!")
            rejected += 1
            transactions.append(["Withdrawal", amount, "Rejected"])
        elif amount > balance:
            print("Insuficient balance!")
            rejected += 1
            transactions.append(["Withdrawal", amount, "Rejected"])
        else:
            balance -= amount
            withdrawals += amount
            transactions.append(["Withdrawal", amount, "Successful"])
            print("Withdrawal successful.")
        else:
            print("Invalid transaction choice!")
            rejected += 1
            transcttsena(["Invalid", amount, "Rejected"])
# Display transaction summary
print("\n========== ATM TRANSACTION SUMMARY ==============")
print("Transaction\t Amount  Status")
for t in transactions:
    print(t[0], "\t", t[1], "\t", t[2])
print("\nTotal Deposits: Rs.", round(deposits, 2))
print("Total Withdrawals: Rs.", round(withdrawals, 2))
print("Rejected Transactions:", rejected)
print("Final Balance: Rs.", round(balance, 2))
print("=============================================")
```

Program Output

```
Enter initial balance: 5000
Enter nnmber of transactions: 4

Transaction 1
1. Deposit
2. Withdraw
Enter your choice: 1
Enter amount: 2000
Deposit successful.

Transaction 2
1. Deposit
2. Withdraw
Enter your choice: 2
Enter amount: 3000
Withdrawal successful.
... (isummary) ...
```



Retail Billing with Multiple Discount Rules

Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

Problem Statement

Write a Python program to process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice. The program should calculate the total amount, tax, discount for each product and display a detailed bill along with the grand total.

Python Code

```
# Retail Billing with Multiple Discount Rules
products = []
total_amount = 0
total_tax = 0
total_discount = 0

n = int(input("Enter the number of products: "))
for i in range(n):
    print("\nEnter details for Product", i + 1)

    name = input("Enter product name: ")
    category = input("Enter category (food/electronics/clothing): ")
    price = float(input("Enter price per unit: "))
    quantity = int(input("Enter quantity: "))

    amount = price * quantity

    # Category-specific tax rates
    if category == "food":
        tax_rate = 5
    elif category == "electronics":
        tax_rate = 18
    elif category == "clothing":
        tax_rate = 12
    else:
        tax_rate = 10

    tax = amount * tax_rate / 100

    # Threshold-based discount
    if  amount >= 5000:
        discount_rate = 15
    elif amount >= 2000:
        discount_rate = 10
    elif amount >= 1000:
        discount_rate = 5
    else:
        discount_rate = 0

    discount = amount * discount_rate  / 100

    # Maximum concession limit
    if discount > 1000:
        discount = 1000

    # Calculate item total
    item_total = amount + tax - discount
    products.append([name, category, price, quantity, amount,
                     tax_rate, tax, discount, item_total])

    total_amount += amount
    total_tax += tax
    total_discount += discount

# Generate itemized invoice
print("\n========== RETAIL INVOICE ==========")
print("Product\tCategory\tQty\tPrice\tAmount\tTax\tDiscount\tTotal")
for p in products:
    print(p[0], "\t", p[1], "\t", p[3], "\t",
          round(p[2], 2), "\t", round(p[4], 2), "\t", "\t",
          round(p[6], 2), "\t", round(p[7], 2), "\t",
          round(p[8], 2))
grand_total = total_amount + total_tax - total_discount
print("\nSubtotal: Rs.", round(total_amount, 2))
print("Total Tax: Rs.", round(total_tax, 2))
print("Total Discount: Rs.", round(total_discount, 2))
print("Grand Total: Rs.", round(grand_total, 2))
print("=" * 40)
```

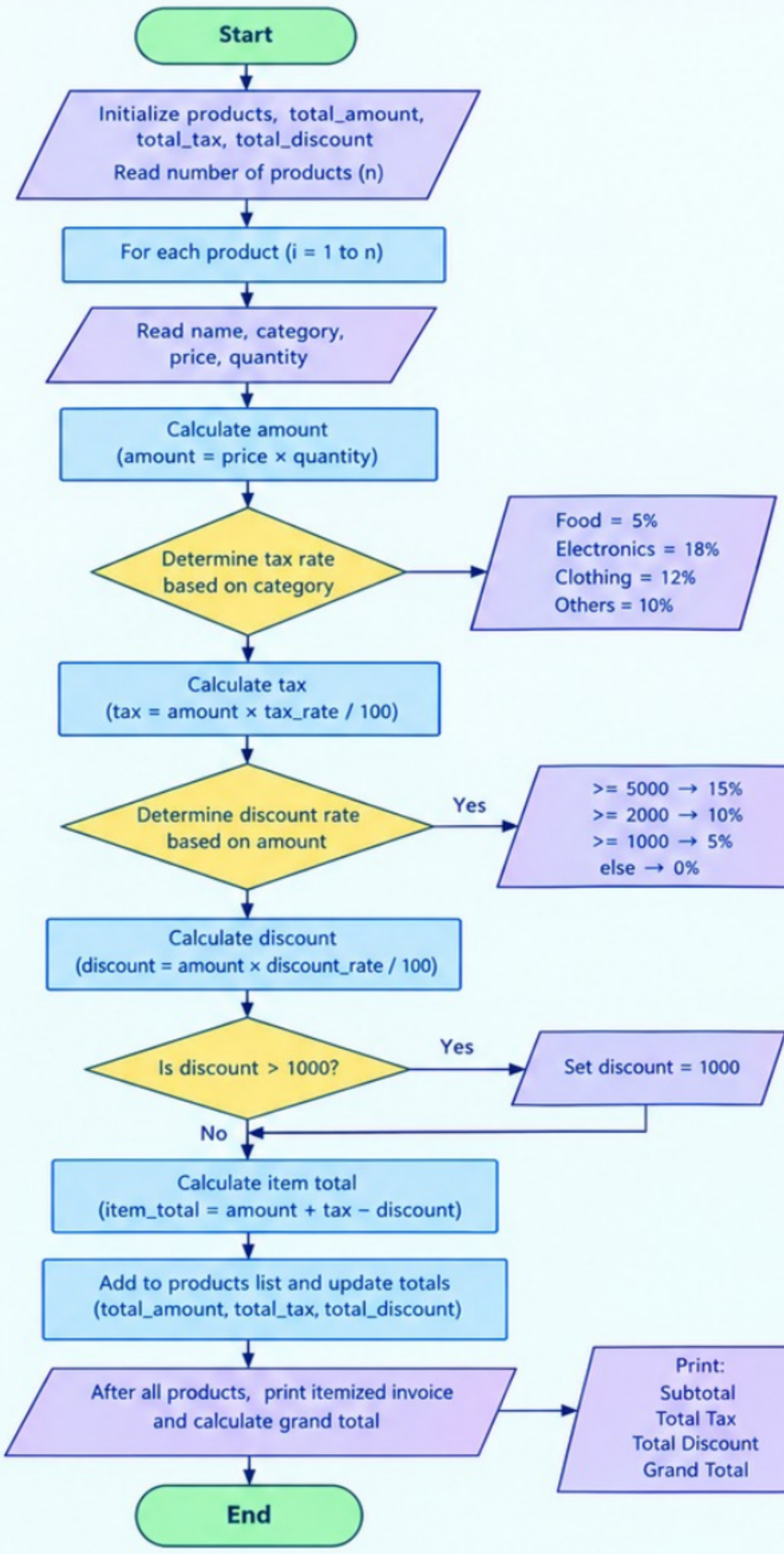
Program Output

```
Enter the number of products: 3
Enter details for Product 1
Enter product name: Rice
Enter category (food/electronics/clothing): food
Enter price per unit: 60
Enter quantity: 5
Enter details for Product 2
Enter product name: Laptop
Enter category (food/electronics/clothing): electronics
Enter price per unit: 50000
Enter quantity: 1
Enter details for Product 3
Enter product name: Shirt
Enter category (food/electronics/clothing): clothing
Enter price per unit: 800
Enter quantity: 3

========== RETAIL INVOICE ==========
Product     Category     Qty    Price     Amount    Tax     Discount    Total
Rice        food         5      60.0      300.0     15.0     15.0       300.0
Laptop      electronics  1      50000.0   50000.0   9000.0   1000.0     19000.0
Shirt       clothing     3      800.0     2400.0    288.0    240.0      2448.0

Subtotal: Rs. 52700.0
Total Tax: Rs. 9303.0
Total Discount: Rs. 1255.0
Grand Total: Rs. 62748.0
========================================
```

Flowchart



Bus Passenger Occupancy Simulation:

Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.

Problem Statement

Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.

Python Code

```
# Bus Passenger Occupancy Simulation
passengers = int(input("Enter initial number of passengers: "))
capacity = int(input("Enter maximum bus capacity: "))
n = int(input("Enter number of stops: "))

if passengers < 0 or capacity <= 0 or passengers > capacity or n <= 0:
    print("Invalid initial input!")
else:
    total_boarded = 0
    total_alighted = 0
    violations = 0
    print("\n========== BUS OCCUPANCY REPORT ==========")

    for i in range(n):
        print("\nStop", i + 1)
        boarded = int(input("Enter number of passengers boarding: "))
        alighted = int(input("Enter number of passengers alighting: "))

        if boarded < 0 or alighted < 0:
            print("Invalid passenger count! Enter non-negative values.")
            continue
        if alighted > pansengers:
            print("Invalid transaction! More passengers cannot alight than are on the bus.")
            continue

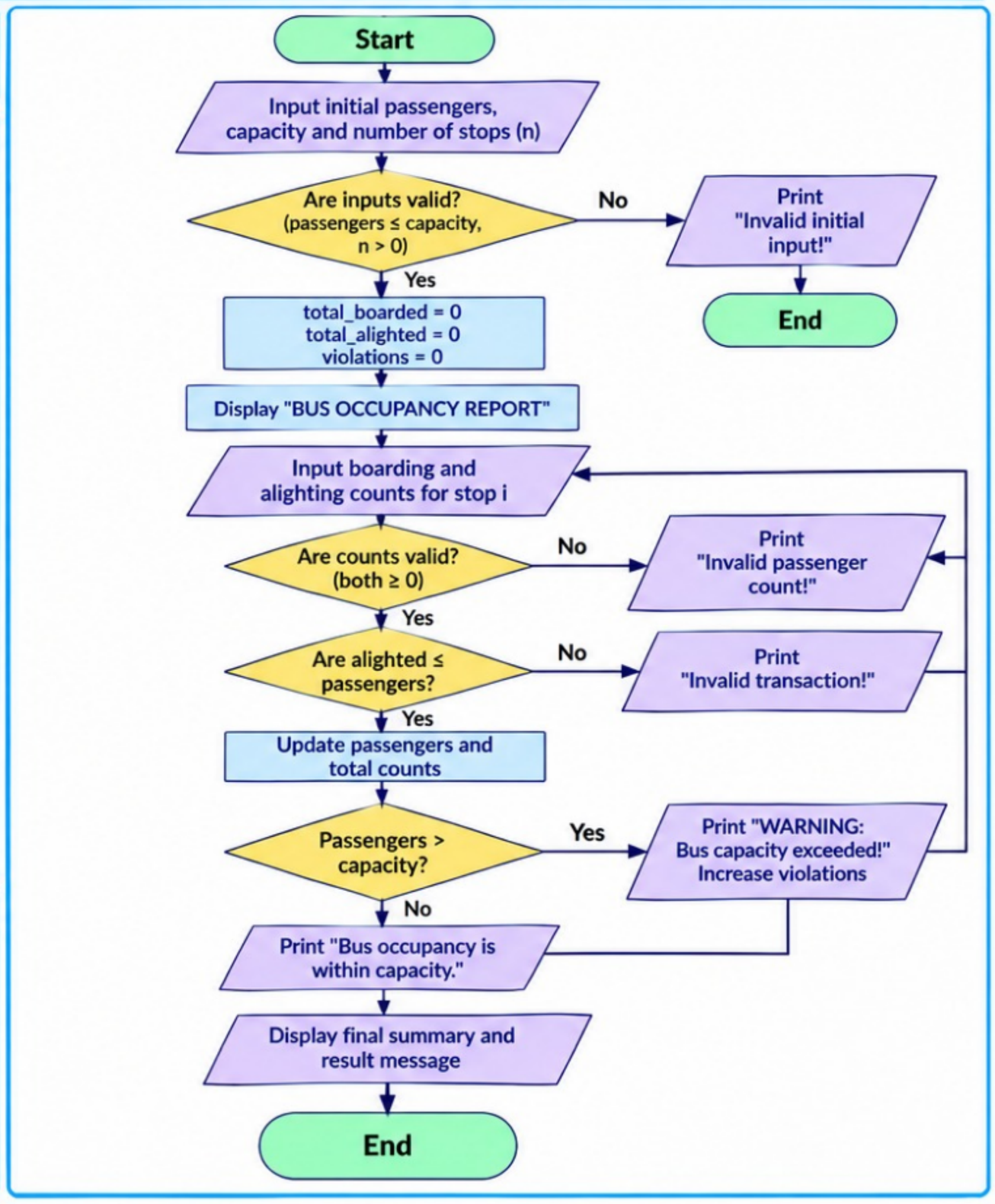
        passengers = passengers - alighted + boarded
        total_boarded += boarded
        total_alighted += alighted
        print("Passengers after stop:", passengers)

        if passengers > capacity:
            print("WARNING: Bus capacity exceeded!")
            violations += 1
        else:
            print("Bus occupancy is within capacity.")
    print("\n========== FINAL SUMMARY ==========")
    print("Total Passengers Boarded:", total_boarded)
    print("Total Passengers Alighted:", total_alighted)
    print("Final Passenger Count:", passengers)
    print("Maximum Bus Capacity:", capacity)
    print("Capacity Violations:", violations)
    if violations > 0:
        print("Result: Capacity violations detected.")
    else:
        print("Result: No capacity violations detected.")
    print("\n==========================================")
```

Program Output

```
========== BUS OCCUPANCY REPORT ==========
Stop 1
Enter number of passengers boarding: 5
Enter number of passengers aligthing: 2
Passengers after stop: 8
Bus occupancy is within capacity.
Stop 2
Enter number of passengers boarding: 4
Enter number of passengers aligthing: 1
Passengers after stop: 11
Bus occupancy is within capacity.

========== FINAL SUMMARY ==========
Total Passengers Boarded: 9
Total Passengers Alighted: 3
Final Passenger Count: 11
Maximum Bus Capacity: 15
Capacity Violations: 0
Result: No capacity violations detected.
```



Food Waste Monitoring System:

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptaable limits.

Problem Statement

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Python Code

```
# Food Waste Monitoring System
days = []
n = int(input("Enter the number of days: "))
waste_limit = float(input("Enter acceptable wastage percentage: "))

total_prepared = 0
total_served = 0
total_leftovers = 0
for i in range(n):
    print("\nEnter details for Day", i + 1)
    day = input("Enter day name: ")
    prepared = int(input("Enter number of meals prepared: "))
    served = int(input("Enter number of meals served: "))
    leftovers = int(input("Enter number of leftover meals: "))

    # Validate meal quantities
    if prepared < 0 or served < 0 or leftovers < 0:
        print("Invalid input! Quantities cannot be negative.")
        continue
    if served + leftovers > prepared:
        print("Invalid data! Served meals and leftovers exceed prepared meals.")
        continue

    # Calculate wastage percentage
    if prepared > 0:
        wastage_percentage = (leftovers / prepared) * 100
    else:
        wastage_percentage = 0

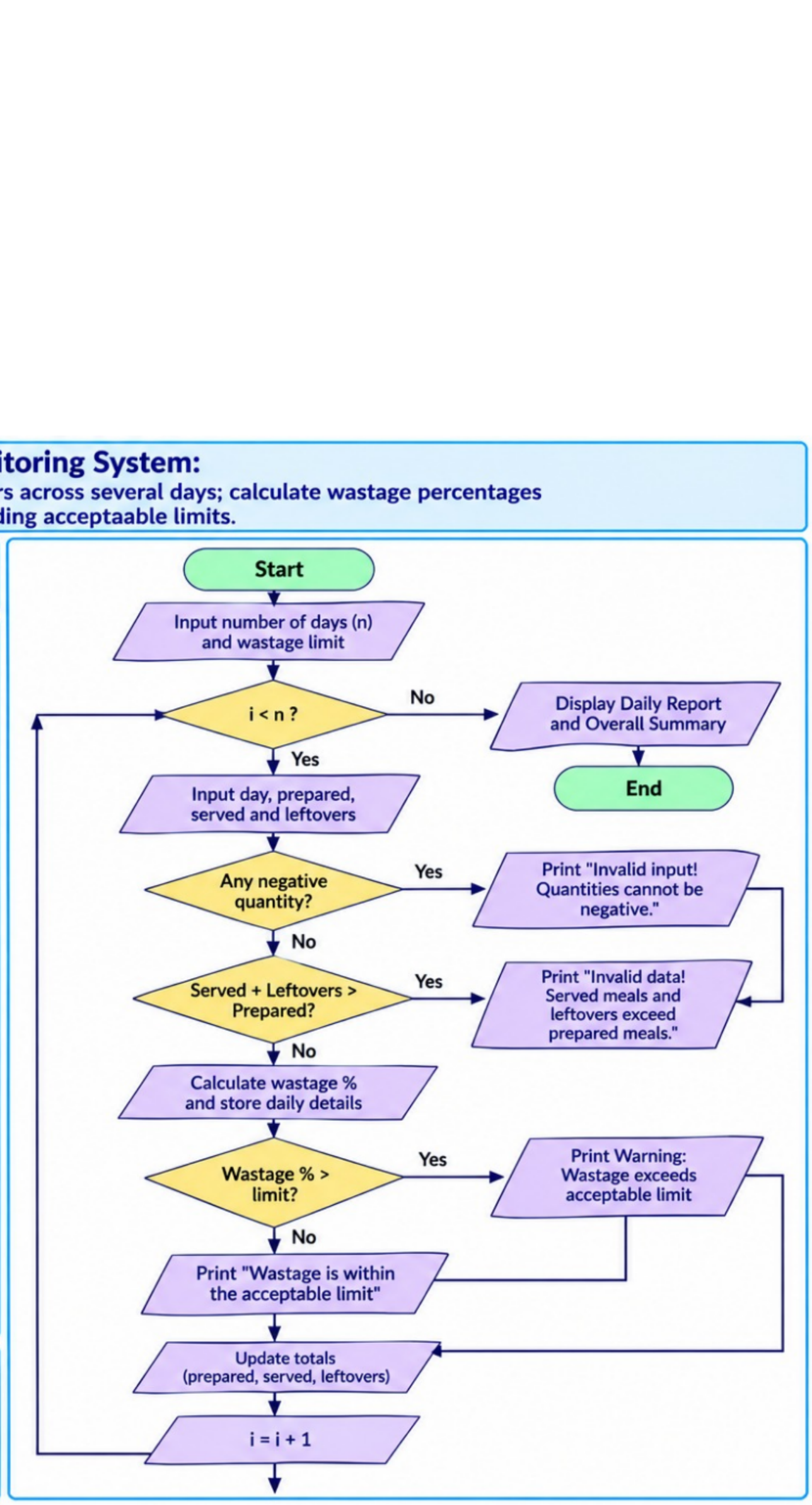
    # Store daily details
    days.append([day, prepared, served, leftovers, wastage_percentage])
    total_prepared += prepared
    total_served += served
    total_leftovers += leftovers

# Display dailly report
print("\n========== FOOD WASTE MONITORING REPORT ==========")
print("Day\t Prepared\t Served\t Leftovers\t Wastage %")
for d in days:
    print(d[0], "\t", d[1], "\t\t", d[2], "\t", d[3], "\t\t", round(d[4], 2), "%")
    if d[4] > waste_limit:
        print("Warning: Wastage exceeds acceptable limit on", d[0])
    else:
        print("Wastage is within the acceptable limit on", d[0])

# Calculate overall wastage percentage
print("\n========== OVERALL SUMMARY ==========")
print("Total Meals Prepared:", total_prepared)
print("Total Meals Served:", total_served)
print("Total Leftover Meals:", total_leftovers)
if total_prepared > 0:
    overall_wastage = (total_leftovers / total_prepared) * 100
    print("Overall Wastage Percentage:", round(overall_wastage, 2), "%")
else:
    print("Overall Wastage Percentage: 0%")
print("=========================================")
```

Program Output

```
==========  FOOD WASTE MONITORING REPORT =========
Day       Prepared      Served     Leftovers      Wastage %
Monday      100          90          10            10.0 %
Tuesday     120          100         20            16.67 %
Warning: Wastage exceeds acceptable limit on Tuesday
========== OVERALL SUMMARY ==========
Total Meals Prepared: 220
Total Meals Served: 190
Total Leftover Meals: 30
Overall Wastage Percentage: 13.64 %
=========================================
```



Text Compression Using Character Runs:

Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial.

Problem Statement

Given a string, compress it using character runs (e.g., "aaabb" → "a3b2"), compare the original and encoded lengths, and determine whether compression is beneficial, provides no reduction, or is not beneficial.

Python Code

```
# Text Compression Using Character Runs
def compress_text(text):
    if not text;
        return ""

    encoded = ""
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1];
            count += 1
        else:
            encoded += text[i - 1] + str(count)
            count = 1

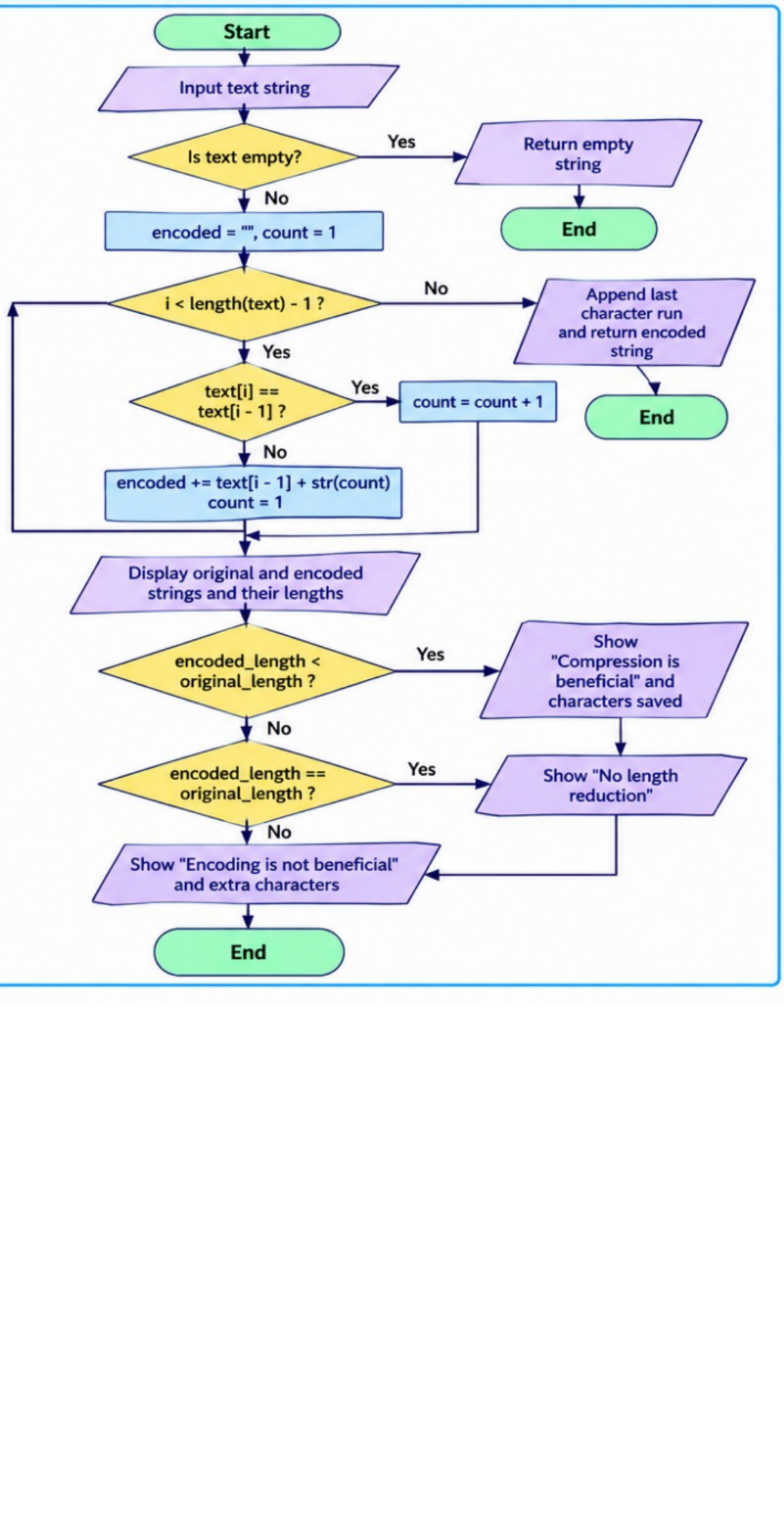
    # Encode the final character run
    encoded += text[-1] + str(count)
    return encoded
# Main program
print("===== TEXT COMPRESSION USING CHARACTER RUNS ======")
text = input("Enter a string: ")
encoded_text = compress_text(text)
original_length = len(text)
encoded_length = len(encoded_text)

print("\nOriginal String:"; text)
print("Encoded String:"; encoded_text)
print("Original Length:"; original_length)
print("Encoded Length:"; encoded_length)
if encoded_length < original_length:
    prin("Result: Compression is beneficial.")
    saved = original_length - encoded_length
    print("Characters Saved:"; saved)
elif encoded_length == original_length:;
    print("Result: Compression provides no length reduction.")
else:
    print("Result: Encoding is not beneficial.")
    extra = encoded_length - original_length
    print("Additional Characters:"; extra)
print("================================================")
```

Program Output

```
===== TEXT COMPRESSION USING CHARACTER RUNS =====
Enter a string: aaabbccccdddde

Original String:  aaabbccccdddde
Encoded String:  a3b2c3d4e1
Original Length: 13
Encoded Length: 8
Result: Compression is beneficial.
Characters Saved: 5
================================================
```



Password Policy Validation Engine:

Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

Problem Statement

Check a given password against the policy rules: at least 8 characters, one uppercase, one lowercase, one digit, one special character, no 3 consecutive identical characters, and no prohibited substrings (e.g., "password", "admin", "123456"). Display the validation result and explain each failed condition.

Python Code

```
# Password Policy Validation Engine
import string

# Password policy rules
MIN_LENGTH = 8
MAX_REPEATED = 2
PROHIBITED_SUBSTRINGS = ["password", "admin", "123456"]

def validate_password(password):
    errors = []

    # Check minimum length
    if len(password) < MIN_LENGTH:
        errors.append("Password must contain at least 8 characters.")

    # Check uppercase letters
    if not any(char.isupper() for char in password):
        errors.append("Password must contain at least one uppercase letter.")

    # Check lowercase letters
    if not any(char.islower() for char in password):
        errors.append("Password must contain at least one lowercase letter.")

    # Check digits
    if not any(char.isdigit() for char in password):
        errors.append("Password must contain at least one digit.")

    # Check special characters
    if not any(char in string.punctuation for char in password):
        errors.append("Password must contain at least one special character.")

    # Check repeated consecutive characters
    for i in range(len(password) - MAX_REPEATED):
        if (password[i] == password[i + 1] == password[i + 2]):
            errors.append("Password must not contain 3 consecutive identical characters.")
            break

    # Check prohibited substrings (case-insensitive)
    lower_password = password.lower()
    for word in PROHIBITED_SUBSTRINGS:
        if word in lower_password:
            errors.append("Password must not contain the prohibited substring: " + word)

    # Display validation result
    if errors:
        print("\nPassword: Invalid")
        for error in errors:
            print("-", error)
    else:
        print("\nPassword: Valid")
        print("All password policy rules are satisfied.")

# Main program
print("========== PASSWORD POLICY VALIDATION ==========")
n = int(input("Enter the number of passwords to check: "))
for i in range(n):
    password = input("\nEnter password " + str(i + 1) + ": ")
    validate_password(password)
print("\n========= VALIDATION COMPLETED =========")
```

Program Output

```
========== PASSWORD POLICY VALIDATION ==========
Enter the number of passwords to check: 2

Enter password 1: password123
Password: Invalid
- Password must contain at least one uppercase letter.
- Password must not contain the prohibited substring: password

Enter password 2: StrongP@ss1
Password: Valid
All password policy rules are satisfied.

========= VALIDATION COMPLETED ==========
```

